



Problema I — Kattis Paintball

Algoritmo de Kuhn
Emparelhamento Máximo
Caminhos Aumentantes

Rafael Da Silva - 2420524
Kayo Nicholas - 2320258
Gustavo Viana - 2320257

Orientador: Prof. Me. Ricardo Carubbi
Disciplina: Resolução de Problemas com Grafos
Centro de Ciências Tecnológicas - CCT
Universidade de Fortaleza - UNIFOR

Paintball: Problema e Modelagem Bipartida

Redução formal para emparelhamento perfeito de tamanho N

Regras e Objetivo

- **Entrada:** N jogadores ($2 \leq N \leq 1.000$) e M pares de visibilidade mútua ($0 \leq M \leq 5.000$).
- **Munição estrita:** Cada jogador possui exatamente **uma bola de tinta**.
- **Condição de tiro:** Só é possível atirar em quem se consegue enxergar ($u \leftrightarrow v$).
- **Alvo exclusivo:** Cada jogador deve ser atingido **exatamente uma vez**.
- **Saída:** Alvo de cada jogador $1 \dots N$ ou `Impossible`.

Propriedade Estrutural: O problema reduz-se a encontrar um **Emparelhamento Perfeito em Grafo Bipartido** (*Bipartite Perfect Matching*), saturando todos os vértices.

A Dualidade de Papéis (2N Vértices)

Elemento Original	Modelo Bipartido Duplicado
Jogador k	Atirador A_k (quem atira) e Alvo B_k (quem recebe)
Vértices (V)	$ V = 2N$ (conjuntos disjuntos A e B)
Visibilidade (u, v)	Duas arestas direcionadas: $A_u \rightarrow B_v$ e $A_v \rightarrow B_u$
Arestas (E)	$ E = 2M$ conexões direcionadas de tiro
Classificação	Grafo Bipartido, simples e não ponderado

Reutilização e Adaptações de algs4

Aderência ao repositório de referência do professor (carubbi/RPG)

Reutilizadas sem Alteração (algs4)

- `algs4.graph.Graph` :
Instanciado com $2N$ vértices para armazenar a adjacência bipartida.
- `algs4.bag.Bag` :
Lista encadeada interna de cada vértice. A inserção em $O(1)$ no início (LIFO) define a ordem de consulta dos alvos.

Por que DepthFirstPaths não serviu?

A classe `DepthFirstPaths` usa marcação permanente para conectividade simples. O emparelhamento exige:

- Marcação temporária (`visitado`) zerada a cada atirador;
- Lógica de **desalocação e realocação recursiva** ao colidir com alvo ocupado.

Implementação: KuhnMatcher

- **Classe dedicada:** criada para implementar o Algoritmo de Kuhn com DFS sobre a estrutura `Graph` .
- **Estado global:** `match_para[alvo]` mantém o emparelhamento construído.
- **Recursão:** tenta realocar o ocupante atual antes de ceder o alvo para o novo atirador.

Versão Autocontida (`main_submissao.py`)

Para o juiz online **Kattis**, unificou-se o modelo em um único arquivo autônomo, substituindo `Bag` por listas nativas do Python e garantindo compatibilidade estrita sem dependências externas.

Algoritmo de Kuhn: Rastreamento no Exemplo C4

Simulação manual detalhada demonstrando o efeito cascata de desalocação

Instância Didática: $N = 4$ jogadores em ciclo (1-2, 2-3, 3-4, 4-1). Devido à inserção LIFO do **Bag**, a ordem de consulta de A_1 é B_4 e depois B_2 .

Passo	Atirador	Alvo Consultado	Situação	Decisão / Ação do Algoritmo	Caminho Aumentante	Emparelhamento Atual
1	A_1 (Jog. 1)	B_4 (Jog. 4)	Livre	A_1 assume B_4 diretamente.	$A_1 \rightarrow B_4$	$\{A_1 \rightarrow B_4\}$
2	A_2 (Jog. 2)	B_3 (Jog. 3)	Livre	A_2 assume B_3 diretamente.	$A_2 \rightarrow B_3$	$\{A_1 \rightarrow B_4, A_2 \rightarrow B_3\}$
3	A_3 (Jog. 3)	B_4 (Jog. 4)	Ocupado por A_1	Efeito Cascata: A_3 pede para A_1 mudar de alvo. A_1 consulta B_2 , que está livre. A_1 migra para B_2 . A_3 assume B_4 .	$A_3 \rightarrow B_4 \rightarrow A_1 \rightarrow B_2$	$\{A_1 \rightarrow B_2, A_2 \rightarrow B_3, A_3 \rightarrow B_4\}$
4	A_4 (Jog. 4)	B_1 (Jog. 1)	Livre	A_4 assume B_1 diretamente.	$A_4 \rightarrow B_1$	$\{A_1 \rightarrow B_2, A_2 \rightarrow B_3, A_3 \rightarrow B_4, A_4 \rightarrow B_1\}$

Resultado Final: Jogador 1 atira no 2 | Jogador 2 no 3 | Jogador 3 no 4 | Jogador 4 no 1. Resposta: **2, 3, 4, 1** (perfeito e bijetor).

Complexidade e Validação Local Rigorosa

Fundamentação de desempenho assintótico e bateria de testes locais

Complexidade Assintótica

Operação	Complexidade	No Pior Caso (Kattis)
Construção do Grafo	$O(N + M)$	2.000 + 10.000 passos
Algoritmo de Kuhn	$O(V \cdot E) = O(NM)$	Limite teórico $\approx 10^7$; na prática $\approx 0,07$ s
Memória do Grafo	$O(N + M)$	< 2 MB de memória
Memória Auxiliar	$O(N)$	Arrays e pilha de chamadas

RecursionError Prevenido

Lição herdada do T1: com $N = 1000$, cadeias lineares de deslocamento poderiam atingir o limite padrão de recursão do Python. Configurou-se preventivamente `sys.setrecursionlimit(200000)`.

Bateria de Testes no Repositório

- Casos Dirigidos (11):**
Exemplos do enunciado (`Sample 1` e `Sample 2 - Impossible`), C4, componentes desconexas, jogador isolado e cadeia com $N = 1.000$.
- Validação Cruzada com Força Bruta:**
3.000 instâncias aleatórias ($N \leq 7$) comparadas a um oráculo que testa todas as $M!$ atribuições: 0 divergências.
- Estresse com gerador.py e Tempo Medido:**
 $N = 1.000$ e $M = 5.000$: $\approx 0,07$ s localmente e 0,14 s no juiz, dentro do limite de 1,0 s.
- Integridade da Atribuição (validador.py):**
Toda resposta é conferida: cada alvo está na adjacência do atirador e cada jogador é atingido exatamente uma vez.

Conclusão e Veredito Oficial no Kattis

Solução validada e aceita pelo juiz online internacional

Evidência Oficial da Submissão (Kattis)

Submission 20629089

Support Kattis

RAFAEL DA SILVA LEITES

Submission 20629089

Edit and resubmit

DATE	PROBLEM	JUDGEMENT	RUNTIME	LANGUAGE	TEST CASES
19:44:14	Paintball	Accepted	0.14 s	Python 3	29/29

Submitted files

Files submitted

Download all (zip)

Mainfile: main_submissao.py

Jump to source for: main_submissao.py | paintball.py

main_submissao.py

Copy

Download main_submissao.py

```
1 """
2 =====
3 Trabalho Prático 2 - Resolução de Problemas com Grafos
4 Problema: Paintball (Kattis - paintball)
5
6 Este arquivo unifica a representação do grafo bipartido
7 e o Algoritmo de Kuhn (busca de caminhos aumentantes via DFS)
8 em um script autossuficiente e otimizado para submissão no
9 juiz online Kattis, sem dependências externas de pacotes.
10 =====
11 """
12
13 import sys
14
15 # Elevação segura do limite de chamadas recursivas para prevenir RecursionError
```

Veredito: Accepted

- Submissão: 20629089
- Problema: Paintball (Kattis)
- Autor: RAFAEL DA SILVA LEITES
- Linguagem: Python 3
- Casos de Teste: 29 / 29 aprovados
- Tempo de Execução: 0.14 s (limite: 1.00 s)
- Arquivo Submetido: main_submissao.py

Conclusão

O modelo de **Grafo Bipartido com Algoritmo de Kuhn** provou-se a estratégia ideal: simplicidade estrutural, alta eficiência e precisão garantida em todos os 29 cenários de teste da plataforma.